

DSA problems

18 March 2026 03:28 PM

11 Group Anagrams Arrays, Strings Medium

Given an array of strings `strs`, group the anagrams together.

You can return the answer in any order.

Example 1:

Input: `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

12 Trapping Rain Water Arrays, Two Pointers, Dynamic Programming, Stacks Hard

Given `n` non-negative integers representing an elevation map

where the width of each bar is `1`, compute how much water

it can trap after raining.

Example 1:

Input: `height = [0,1,0,2,1,0,1,3,2,1,2,1]`

Output: `6`

10 Number of Islands Graphs, Arrays Medium

Given an `m x n` 2D binary grid `grid` which represents a map of `'1'`'s (land) and `'0'`'s (water), return *the number of islands*.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
```

Output: `1`

9 Valid Parentheses Strings, Stacks Easy

Given a string `s` containing just the characters

`'('`, `)'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the

correct order.

3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: s = "()"

Output: true

8 Longest Substring Without Repeating Characters

Strings,
Two
Pointers,
Sliding
Windows

Medium

Given a string `s`, find the length of the longest substring without duplicate characters.

Example 1:

Input: s = "abcabcbb"

Output: 3

1 Text Justification

Given a sequence of words and a line width, format the text to fit within the specified width. You must justify the text, distributing extra spaces evenly between words.

Very likely

Hard

2 Check if Every Row and Column Contains All Numbers

Determine if every row and every column in a given square matrix contains all the numbers from 1 to n, where n is the size of the matrix.

Effectively, check if it's a valid Sudoku (without subgrids).

Very likely

Easy

3 Find Words That Can Be Formed by Characters

You're given a list of words and a string of characters.

Determine which words can be fully formed using only the characters from the given string, and return the sum of their lengths.

Somewhat likely Easy

6. Merge Intervals

Arrays, Greedy Algorithms Medium

Given an array of intervals where `intervals[i] = [starti, endi]`, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

Input: intervals = [[1,3],[2,6],[8,10],[15,18]]

Output: [[1,6],[8,10],[15,18]]

7. Best Time to Buy and Sell Stock

Arrays, Dynamic Programming Easy

You are given an array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: `5`

4. Two Sum

Arrays Easy

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to `target`*.

You may assume that each input would have *exactly* one solution, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

5. LRU Cache

Linked Lists Medium

Design a data structure that follows the constraints of a [Least Recently Used \(LRU\) cache](#).

Implement the `LRUCache` class:

- `LRUCache(int capacity)` Initialize the LRU cache with positive size `capacity`.
- `int get(int key)` Return the value of the `key` if the `key` exists, otherwise return `-1`.
- `void put(int key, int value)` Update the value of the `key` if the `key` exists. Otherwise, add the `key-value` pair to the cache. If the number of keys exceeds the `capacity` from this operation, evict the least recently used key.

The functions `get` and `put` must each run in $O(1)$ average time complexity.

Example 1:

Input

`["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]`

`[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]`

Output

`[null, null, null, 1, null, -1, null, -1, 3, 4]`